PatoSim

Novidades de outubro e novembro de 2026 e guia de uso

navegação leve, World Builder, sensores subaquáticos e instalação em outras máquinas

PDI-DL – `https://github.com/PDI-DL/PatoSIM` (branch `main`)

9 de novembro de 2026

Resumo

O PatoSim é uma extensão do Isaac Sim 5.1 para simular um ROV (BlueROV) e gravar dados de câmera, sonar, LiDAR, DVL e barômetro. Ele junta a estrutura de gravação do MobilityGen (registro de robôs e cenários, gravação em duas fases, `Reader`/`Writer`) aos sensores subaquáticos do OceanSim. O que mudou:

- **Navegação leve** (**novo**): todos os sensores começam desligados. Pilotar o ROV pelo mundo fica fluido, e cada gravação liga só os sensores de que precisa (seção 2.2).
- **World Builder** (**novo**): um segundo launcher, sem robô, que carrega o mundo, insere objetos do dataset e confere/aplica as classes semânticas, a refletividade do sonar e as colisões antes da gravação (seção 3).
- **Sensores**: sonar no modo `raw` (mantém a queda real de intensidade com a distância) e salvo durante a gravação; LiDAR com alcance e atenuação de scanners subaquáticos; teleoperação por gamepad em 6 graus de liberdade; extrínsecos dos sensores gravados; mapa de ocupação em camadas de profundidade (seção 4).
- **Funciona em outras máquinas** (**novo**): o código do OceanSim agora vem sempre do próprio repositório (`omni.ext.patosim.oceansim`). Antes ele podia vir de uma cópia antiga em `app/extsUser`. Foram removidos os caminhos fixos de uma máquina (assets, mundo de exemplo, link `isaacsim/oceansim`, testes).
- **Instalado e testado na pdi_mini_3** (RTX 5060 Ti) com o mesmo procedimento do README. Nela e nesta máquina (RTX 4070) passam os 32 testes unitários e os testes de integração com o Isaac, exceto o do sinal do sonar (seção 5).

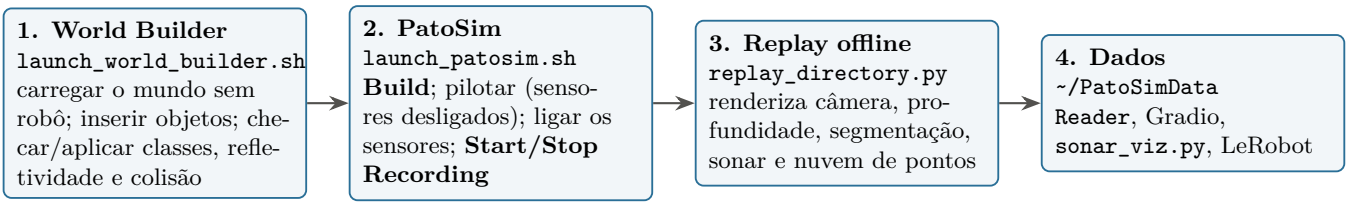


Figura 1: Fluxo de trabalho. O passo 1 é opcional para mundos já preparados; o Build do passo 2 também aplica colisão e refletividade padrão.

1 Instalação

1.1 Máquina nova

Pré-requisitos: Ubuntu com GPU NVIDIA RTX e o **Isaac Sim 5.1.0** (pacote standalone extraído, por exemplo em `~/isaacsim`). Os comandos rodam na raiz do repositório.

```
git clone https://github.com/PDI-DL/PatoSIM.git PatoSim && cd PatoSim
./link_app.sh --path ~/isaacsim                # cria app/ -> Isaac Sim
./app/python.sh -m pip install tqdm wheel       # dependências no Python do Isaac
cd path_planner
../app/python.sh -m pip install pybind11
../app/python.sh -m pip install --no-build-isolation -e .   # planejador em C++
cd ..
python3 -m pip install huggingface_hub hf_transfer        # qualquer Python 3
python3 scripts/assets/downloadAll.py                     # assets -> assets/models
```

Os assets (mundos, modelo do BlueROV e objetos do dataset) estão no dataset público `PDI-DL/PatoSimAssets` do Hugging Face. O `downloadAll.py` os coloca em `assets/models/`. O `scripts/assets/assets.sh` trabalha com um clone git do dataset privado `PDI-DL/PDI_3DUW` e precisa de login com acesso a ele. O PatoSim também os encontra direto em `assets/` (`worlds/`, `bluerov/`, `plataforms/`, `statues_temples/`, `scenario/`), que é o layout da pdi_mini_3.

1.2 Atualizar uma máquina que já tem o PatoSim

Foi o que se fez na pdi_mini_3 em 09/11/2026. A cópia dela estava na branch `update_imports`, que agora faz parte do `main`. Os assets em `assets/`, fora do git, não são tocados.

```
cd ~/Documents/PatoSIM
git fetch origin && git switch main && git pull --ff-only
./app/python.sh -m pip install tqdm wheel
cd path_planner && ../app/python.sh -m pip install pybind11 \
  && ../app/python.sh -m pip install --no-build-isolation -e . && cd ..
```

O `path_planner` é recompilado no último comando. Confira com os testes da seção 5 ou com

```
./app/python.sh -c "import tqdm, mobility_gen_path_planner; print('ok')"
```

1.3 Abrir

```
./scripts/launch_patosim.sh          # simulador: robô, sensores, gravação
./scripts/launch_world_builder.sh    # World Builder: só o mundo, sem robô
```

Variáveis opcionais: `PATOSIM_DATA` (pasta dos dados, padrão `~/PatoSimData`), `PATOSIM_DEV_SCENE` (mundo do botão **Use MHL Scene**, padrão `worlds/Prototipo1/world.usd` dos assets) e `PATOSIM_ROV_WAYPOINTS` (arquivo de waypoints do cenário de waypoints).

2 Usar o simulador

2.1 Montar o cenário

Na janela **PatoSim**:

1. **Use MHL Scene** preenche **USD Path / URL** com o mundo de exemplo. Para outro mundo, cole o caminho do `.usd`.
2. **Scenario Type**: `OceanSimROVTeleoperationScenario` (teclado), `OceanSimROVGamepadTeleoperationScenario`, `OceanSimROVWaypointScenario` ou `OceanSimROVPathFollowingScenario`. **Robot Type**: `OceanSimROVRobot`.
3. Em **Robot Setup**, marque os sensores (se for gravar) e ajuste velocidades e profundidade.
4. **Build**. Em alguns segundos aparecem o mundo, o ROV e o mapa de ocupação. **Reset** reposiciona o robô.

Botão / campo	O que faz
Build, **Reset**	monta o cenário com o mundo, o robô e os sensores marcados; reposiciona o robô
Start Recording / **Stop Recording**	grava em `~/PatoSimData/recordings/<data-hora>/`
Robot Setup	**Sensors** (Front Camera, Stereo, LiDAR, Sonar, DVL, Barometer), Lin/Ang Speed, **Depth (m)**, DVL Debug e **Sonar Refl.** (aplica refletividade aos meshes do mundo no Build)
Recording Settings	estado da gravação, nome e duração, pasta de saída
Window Manager	mostra/esconde as janelas: Sensor Preview (câmera e leque/grade do sonar, com intervalos de atualização), LiDAR Preview (vista de cima), Occupancy Map, Path Planning, Dataset Object
Dataset Object	escolhe um objeto do dataset para inserir no mundo no Build
Replay Config	opções do replay offline

Tabela 1: Controles principais da janela PatoSim.

2.2 Sensores e navegação leve

Ao abrir, **nenhum sensor está ligado**: o ROV é montado só com o corpo e a física, sem renderização de câmera nem raycast de sonar e LiDAR. Para gravar dados, marque em **Robot Setup** → **Sensors** só o que a sessão precisa e clique **Build** de novo. Os sensores não entram no laço de controle, então pilotar sem eles não muda a navegação.

Sensor	Gera	Observação
Front Camera, Stereo	RGB subaquático (e cru), profundidade e segmentação no replay	par estéreo opcional
Sonar (imaging sonar)	intensidade polar e imagem em leque; `state/sonar/raw` durante a gravação	modo `raw` por padrão
LiDAR (RTX)	nuvem de pontos cortada no alcance do perfil e atenuada	perfil `insight_micro` (0,13 a 7 m)
DVL	velocidade em relação ao fundo	**DVL Debug** desenha os feixes
Barometer	pressão e profundidade	

Tabela 2: Sensores do `OceanSimROVRobot`.

2.3 Pilotar

Teclas	Movimento	Teclas	Movimento
`W` / `S`	frente / trás	`J` / `L`	girar à esquerda / direita (yaw)
`A` / `D`	esquerda / direita	`I` / `K`	arfagem (pitch)
↑ / ↓	subir / descer	← / →	rolagem (roll)

Tabela 3: Teclado no `OceanSimROVTeleoperationScenario`. A ação do ROV é força e torque no corpo $[F_x, F_y, F_z, T_x, T_y, T_z]$.

No `OceanSimROVGamepadTeleoperationScenario`, o analógico esquerdo comanda frente/lado e o direito comanda subir/descer e yaw, com zona morta e curva *expo* (`rov_gamepad_deadzone`, `rov_gamepad_expo` no `Config`). Se o gamepad desconectar, o comando vai a zero.

2.4 Navegação autônoma

- `OceanSimROVPathFollowingScenario`: sorteia objetivos 3D, planeja o caminho em XY no mapa de ocupação com perfil linear de profundidade, segue com *pure pursuit* e replaneja a cada objetivo. Os parâmetros (`rov_path_xy_speed`, `rov_path_z_speed`, `rov_path_yaw_gain`, `rov_path_lookahead`, `rov_path_goal_threshold`, `rov_path_z_min/z_max`) são atributos da classe em `scenarios.py`. A janela Path Planning ainda não os edita.
- `OceanSimROVWaypointScenario`: segue os waypoints de `exts/omni.ext.patosim/demo/demo_waypoints.txt` (uma linha `x y z qw qx qy qz`) ou do arquivo em `$PATOSIM_ROV_WAYPOINTS`.

2.5 Gravar, renderizar e ver os dados

A gravação é feita em duas fases. Durante a simulação são salvos as poses, as ações, os estados dos sensores e o sonar bruto. As imagens são renderizadas depois, offline, com o Isaac fechado:

```
python scripts/replay_directory.py --render_interval=20   # 1 quadro a cada 20 passos de física
python examples/04_visualize_gradio.py                     # navegador em http://127.0.0.1:7860
python scripts/sonar_viz.py --data ~/PatoSimData/recordings/<gravacao>
```

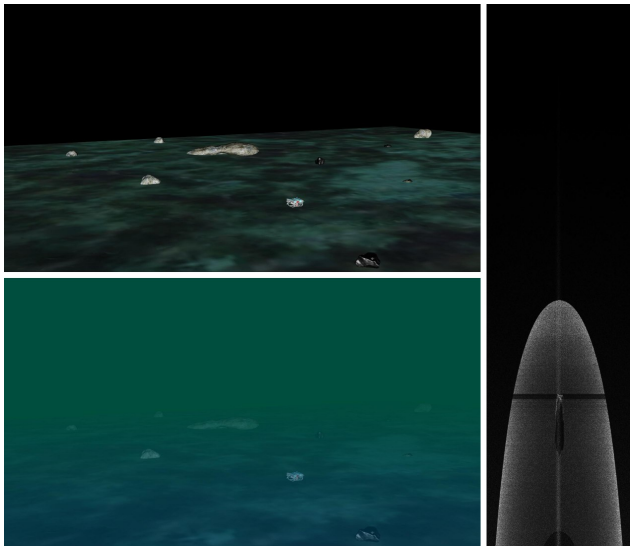


Figura 2: Saídas de uma gravação de 24/03/2026 com o mundo Prototipo1: câmera sem o efeito da água (em cima), a mesma imagem com o modelo subaquático (embaixo) e a imagem em leque do sonar (direita).

Cada gravação é uma pasta com `config.json`, `stage.usd`, `sensors_extrinsics.json` (**novo** desde 05/10: transformação fixa corpo→sensor de cada sensor ativo), `occupancy_map/`, `occupancy_map_3d/` (no modo `3d_stack`) e `state/` (`common/`, `sonar/raw/` e, depois do replay, `rgb/`, `depth/`, `segmentation/`, `normals/`). Poses ficam no referencial do mundo, em metros, com quatérnio na ordem `[w, x, y, z]`.

3 World Builder

`./scripts/launch_world_builder.sh` abre o Isaac só com a extensão `omni.ext.patosim_worldbuilder`, sem robô, sensores ou gravação. Ela usa as mesmas funções de `omni.ext.patosim.build` que o simulador usa, então o que ela mostra vale para o Build.

Seção / botão	O que faz
Load World (no robot)	carrega o `.usd` em `/World/scene`, sem robô
Import Dataset Object	lista `plataforms/`, `statues_temples/`, `scenario/`; posição, rotação, escala e refletividade; **Insert Object Into World**
Check Semantic Classes	conta os meshes sem os rótulos `class` e `reflectivity` que o sonar usa (sem eles o sonar usa um valor padrão, que parece certo na imagem mas está errado para o dataset)
Apply Default Reflectivity	aplica a refletividade padrão aos meshes que não têm
Check Physics	conta a geometria sem colisão (invisível para os raycasts do sonar e do LiDAR)
Apply Physics (Collisions)	aplica colisão no mundo inteiro

Tabela 4: World Builder. Nos testes, o Prototipo1 tinha 428 meshes sem classe nem refletividade e 234 com colisão. Depois dos dois Apply, ficaram 428/428 com refletividade e 428/428 com colisão.

As correções valem para o stage aberto. O **Build** do simulador já aplica colisão no mundo todo e, com **Sonar Refl.** marcado, a refletividade padrão. O World Builder serve para ver antes o que falta num mundo, por exemplo meshes sem classe semântica, e corrigir o próprio arquivo do mundo.

4 Novidades em detalhe

Novidade	O que muda	Como usar
09/11/2026		
OceanSim dentro da extensão	imports `omni.ext.patosim.oceansim`; o sonar não depende mais de uma cópia em `app/extsUser` (na pdi_mini_3 essa cópia era de março)	nada a fazer; o teste confere de onde vem o código
Assets em dois layouts	`assets/models/` ou `assets/`	qualquer um dos dois
Mundo de exemplo	resolvido a partir do repositório	**Use MHL Scene** ou `PATOSIM_DEV_SCENE`
24/10/2026		
Navegação leve	sensores desligados por padrão	marcar em **Robot Setup** antes de gravar
World Builder	launcher sem robô com checagens	`launch_world_builder.sh` (seção 3)
Testes de integração	cenário ROV completo, gravação curta e World Builder com o Isaac headless	seção 5
05/10/2026		
Sonar `raw`	sem normalização por anel/quadro: a intensidade cai com a distância como no sensor real	padrão; `range`/`all` continuam como opções de visualização
Sonar bruto na gravação	`state/sonar/raw/*.npy` gravado ao vivo, não só no replay	ligar o Sonar
LiDAR subaquático	perfis `insight_nano` (0,13–2,5 m), `insight_micro` (0,13–7 m) e `insight_pro` (0,5–15 m), com probabilidade de retorno $e^{-\alpha d}$	`lidar_range_profile` e `lidar_attenuation_coeff` no `Config`
Gamepad 6 DOF	novo cenário, com zona morta e expo	`OceanSimROVGamepadTeleoperationScenario`
Extrínsecos	`sensors_extrinsics.json` em cada gravação	combinar com `robot.position` e `robot.orientation`
Mapa em camadas	`OccupancyMapStack`: uma fatia 2D por banda de profundidade (Fig. 3)	`occupancy_map_mode="3d_stack"` no `Config`; a navegação ainda usa a fatia única
Escala de assets	compensa `metersPerUnit` do asset (assets em cm não ficam 100× maiores)	automático ao inserir objetos

Tabela 5: Novidades, da mais recente para a mais antiga.



Figura 3: Mapa de ocupação em camadas do Prototipo1, gerado no teste de integração: fatias centradas em $z = -4$, -2 e 0 m (preto = ocupado). O fundo desse mundo fica em $z \approx 0$ m. Os corais atravessam o fundo e aparecem na fatia de -2 m. A fatia de 0 m corta o próprio fundo.

5 Testes

```
python3 -m pytest tests/unit                                          # 32 testes, sem Isaac
./app/python.sh tests/integration/run_gpu_integration_checks.py       # Isaac headless, 1 a 3 min
./app/python.sh tests/integration/run_worldbuilder_extension_check.py # Isaac headless, < 1 min
```

O teste de integração monta um cenário ROV com todos os sensores, grava uma sessão curta e confere os arquivos. Ele também testa as ferramentas do World Builder, o mapa em camadas e o gamepad, e confirma que o código do sonar vem do repositório. A comparação com uma gravação antiga só roda se ela existir

(PATOSIM_LEGACY_RECORDING).

Checagem	Esta máquina RTX 4070, driver 565	pdi_mini_3 RTX 5060 Ti, driver 580
Testes unitários (`tests/unit`)	32/32	32/32
Extensões, imports e código do sonar vindo do repositório	ok	ok
`config.json` antigo (de antes dos campos novos)	ok	pulado (sem a gravação)
World Builder: checar e aplicar classes, refletividade e colisão; listar e inserir objetos	ok (428 meshes, 11 objetos)	ok (429 meshes, 12 objetos)
Cenário ROV com os 5 sensores, sonar `raw`, perfil `insight_micro` do LiDAR	ok	ok
Sinal do sonar (intensidade diferente de zero)	**falha**	**falha**
Gravação ao vivo: extrínsecos, `state/common`, sonar bruto, imagens do sonar	ok	ok
Mapa em camadas (3 fatias, salvo em disco)	ok	ok
Gamepad sem controle físico (comando zero)	ok	ok
Carregar gravação antiga no mesmo processo	crash do Isaac	pulado
Extensão World Builder sozinha (sem a principal, janela criada)	6/6	6/6
Total do teste de integração	25 ok, 1 falha, 1 crash	26 ok (2 pulados), 1 falha

Tabela 6: Resultados de 09/11/2026 com o código do `main` (`8c92c04`). A falha do sonar e o crash estão em Limitações conhecidas.

6 Limitações conhecidas

- **Spawn**: `_compute_safe_underwater_spawn()` (`build.py`) leva o ROV para 1 m acima do objeto mais alto do mundo sempre que o ponto configurado cai dentro da caixa que envolve o mundo, o que é o caso normal. No Prototipo1 o fundo fica em $z \approx 0$ m e os objetos vão até 5,7 m. O robô nasce em $z = 6{,}7$ m, e a profundidade padrão de **Depth (m)** (-2 m) fica abaixo do fundo. Até a correção, posicione o robô com as teclas ou ajuste a profundidade ao mundo.
- **Sonar no teste headless**: o teste de integração monta o sonar, roda o kernel e grava os arquivos, mas a intensidade sai zerada (mínimo = máximo = 0), nas duas máquinas. Isso acontece mesmo com o ROV 1,5 m acima da primeira superfície abaixo dele (nesta máquina, o fundo do Prototipo1) e um objeto 3 m à frente. Pela interface o sonar funciona (figura da seção 2, gravação de 24/03). A causa no modo headless não foi isolada. Antes de uma gravação longa, confira o sonar na janela Sensor Preview.
- **Checagem com a gravação antiga**: ao carregar a gravação de 24/03 no mesmo processo dos outros testes, o Isaac cai (crash nativo em `omni.usd` ao criar o `World` de novo). As outras checagens já estão salvas nesse ponto. Em máquinas sem essa gravação a checagem é pulada.
- **Path Planning**: a janela edita só o cenário terrestre herdado do MobilityGen. Os parâmetros do `OceanSim ROVPathFollowingScenario` ficam no código.
- `oceansim/utils/asset_path.json` aponta para `/mnt/external/isaac/OceanSim_assets`, que só existe numa máquina. Ele é só um caminho alternativo para o modelo do ROV (o principal vem dos assets do repositório). Para mudar: `scripts/register_oceansim_assets.sh <pasta>` ou `OCEANSIM_ASSET_PATH`.
- As cópias `OceanSim`, `OceanSim.bak` e `PatoSIM` em `app/extsUser` não são mais usadas pelo PatoSim e podem ficar onde estão.